Plan de Game Day para evaluar la resiliencia de mi-microservicio

Manuel Fernando Santofimio Tovar y Beycy Yuliet Rojas

Maestría en Arquitectura de Software, Universidad de La Sabana

EFMAS-OBAP20265: Observabilidad en Ambientes Productivos

Actividad 3.2. Laboratorio Plan de Game Day

Carol Daniela Suarez Mendez

18 de octubre de 2026

Plan de Game Day para evaluar la resiliencia de mi-microservicio

Se diseña un ensayo controlado para refutar hipótesis del estado estable y convertir debilidades en mejoras verificables (Principles of Chaos Engineering, 2019). La actividad exige tres diseños y una ejecución real (Universidad de La Sabana, 2026).

Arquitectura y estado estable

El objetivo es FastAPI en app/app.py del laboratorio anterior (yuliet719-beep, s. f.; commit 7d4e7f1913f2). Expone /hello, /health y /metrics; depende de Python, Uvicorn e instrumentación Prometheus. No utiliza base de datos ni HTTP externo. GitHub Actions, Jenkins, Docker, Helm, Argo CD y Kubernetes integran la entrega; no se perturban.

El cliente accede a target:8000 por 127.0.0.1:18082; Prometheus consulta /metrics cada 2 s. Se conserva el código, pero la imagen distroless se sustituye por python:3.11-slim con iproute2: una réplica, 1 CPU, 256 MiB y NET_ADMIN solo en su red. El estado estable exige 5 solicitudes/s, disponibilidad ≥99% y p95 <200 ms, timeout de 2 s, sin reintentos y referencia de 30 s.

El director controla inicio/parada; el operador inyecta/revierte; el observador registra SLI. Antes del fallo se verifica salud, UP y referencia válida; se aborta ante diez fallos consecutivos. La Tabla 1 presenta hipótesis previas cuya refutación es válida.

Tabla 1

Hipótesis medibles de conservación del estado estable

ID	Hipótesis
H1	En estado estable, cuando se añaden 300 ms de latencia de salida a target hasta 60 s, esperamos disponibilidad ≥99%, p95 <200 ms y retorno a ese umbral en 30 s tras el rollback.
H2	En estado estable, cuando un proceso consume la cuota de una CPU durante 60 s, esperamos disponibilidad ≥99%, p95 <200 ms y recuperación a ambos umbrales en 30 s.
H3	En estado estable, cuando se pierde la única réplica durante 20 s, esperamos disponibilidad ≥99% durante el fallo y retorno de respuestas 2xx en ≤30 s desde su restauración.

Diseño experimental y observabilidad

Cada fallo se limita a target, sin red del host ni producción. La Tabla 2 alinea los diseños con H1-H3. E1 usa netem, que altera paquetes fuera del handler (Ludovici & Pfeifer, 2011). Los 60 s permiten hasta 300 solicitudes y 30 scrapes; los 20 s de E3 acotan la caída. Se miden 30 s antes y después.

Tabla 2

Fallos, alcance y recuperación automática

Diseño y fallo	Radio y duración	Inyección y rollback
E1/H1: latencia. Ejecutado.	Solo eth0 de target, incluidos /hello y scrape. Máximo 60 s.	tc qdisc replace dev eth0 root netem delay 300ms. Watchdog interno elimina qdisc a 60 s; finally lo retira al abortar. Se verifica noqueue.
E2/H2: agotamiento CPU. Diseñado.	Solo cuota de target; Prometheus queda fuera. Carga de 60 s.	stress-ng --cpu 2 --timeout 60s, con supervisor timeout externo TERM a 60 s y KILL a 65 s si persiste. Solo termina trabajadores del ensayo.
E3/H3: caída de proceso. Diseñado.	Única réplica target, hasta 20 s; no es partición de red.	Armar watchdog host independiente que inicie target a 20 s; aplicar compose kill -s SIGKILL target. Finally inicia de nuevo. Verificar salud y UP.

Gremlin modelaría CPU/latencia; Litmus, sondas y selección sandbox; Chaos Monkey, pérdida de instancia. Son equivalencias conceptuales. README.md concreta los runbooks. Se ejecuta un fallo a la vez; se detiene ante degradación residual.

SLI, registros y trazas

Para H1-H3: disponibilidad = 2xx/total; error = 1 - disponibilidad; throughput = éxitos/segundos; p95 por rango más cercano, incluidos timeouts. El CSV registra UTC, fase, estado, duración y trace_id cliente. H1 contrasta red/handler; H2 observa CPU y memoria con docker stats; H3, conexión, up y reinicio. Los histogramas permiten estimar percentiles (Prometheus Authors, s. f.).

Uvicorn aporta accesos; eventos.json, inyección/retirada; /metrics y up, proceso/scrape. Los IDs son spans cliente sin traza distribuida servidor; se propone OpenTelemetry y logs correlacionados. La recuperación se valida en 30 s, sin estimar MTTR exacto ni SLO mensual.

Resultados y acciones de mejora

Se ejecutó E1 el 18 de octubre de 2026, 22:05-22:06 (Colombia). Se inició compose.yaml y ejecutar.py: referencia, watchdog, netem, parada y recuperación. Las 336 solicitudes y los comandos están en evidencias. La Tabla 3 contrasta H1 (≥99% y p95 <200 ms) con los resultados.

Tabla 3

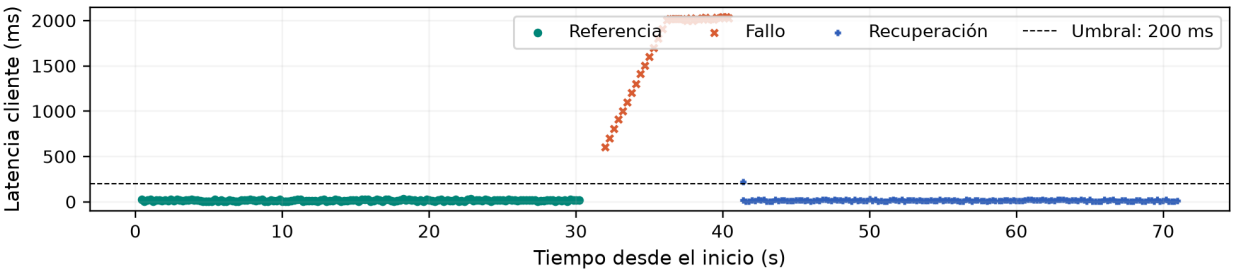
Resultados medidos antes, durante y después de E1

Fase	Total/2xx	Disponibilidad	p95 (ms)
Referencia	150/150	100.00%	26.5
Fallo	36/15	41.67%	2038.1
Recuperación	150/150	100.00%	26.2

H1 se refutó: 21 timeouts y parada a 9.03 s, con 1.66 éxitos/s. Finally retiró netem y tc mostró noqueue. El preensayo abortado se conserva aparte; E2/E3 siguen como diseños. La Figura 1 muestra la degradación y recuperación.

Figura 1

Latencia de las solicitudes del ensayo E1



Nota. Elaboración propia desde solicitudes.csv; los timeouts incluyen la espera.

Debilidad sistémica y remediación

La única réplica es vulnerable a la red: /health no garantiza experiencia de usuario. TCP y Docker pueden amplificar el retardo; no se atribuye todo a 300 ms. Las alertas previas a 1 y 5 min no aseguran detectar este fallo breve.

Prioridad 1: el observador añadirá sondeo externo con detección ≤10 s; el operador probará dos réplicas con éxito ≥99% en E3. Prioridad 2: ajustar timeouts y capacidad con E2 y correlacionar trazas. El director repetirá el plan para cerrar mejoras SRE con SLI.

Referencias

Ludovici, F., & Pfeifer, H. P. (2011, 25 de noviembre). *tc-netem(8): Linux manual page*. man7.org. https://man7.org/linux/man-pages/man8/tc-netem.8.html

Principles of Chaos Engineering. (2019, marzo). *Principles of chaos engineering*. https://principlesofchaos.org/

Prometheus Authors. (s. f.). *Histograms and summaries*. Prometheus. https://prometheus.io/docs/practices/histograms/

Universidad de La Sabana. (2026). *Actividad 3.2. Laboratorio Plan de Game Day U3* [Consigna y criterios de evaluación]. VirtualSabana. https://unisabanae-learning.unisabana.edu.co/mod/assign/view.php?id=306420

yuliet719-beep. (s. f.). *mi-microservicio* [Código fuente, versión 7d4e7f1913f2]. GitHub. https://github.com/yuliet719-beep/mi-microservicio/tree/7d4e7f1913f26262ff23f9deee14ebbfe2b3f797

Anexos

Anexo A. Repositorio y evidencias reproducibles

Repositorio: https://github.com/yuliet719-beep/mi-microservicio

Carpeta de entrega: Actividad 3.2 - Laboratorio Plan de Game Day. Contiene el informe, los datos originales, la captura de Prometheus y los archivos para repetir el ensayo.

Archivo complementario: Evidencias_Game_Day.zip. Incluye evidencias, plan previo, código de ejecución, configuración del sandbox y README con la auditoría de los 20 criterios.